# Day 7 Revision Notes: Python SQLite Connection & Project 2

Topic: Integrating Python with SQLite (sqlite3), Cursor Execution, and CRUD Operations | Date: 2026-10-11

## 1. Python & SQLite Architecture

In Day 7, we upgraded our expense storage architecture from flat JSON files to a relational database using Python's built-in **sqlite3** module. Unlike JSON files which require parsing the entire dataset into RAM, SQLite reads and writes fixed 4KB binary pages on disk.

| Feature / Dimension | JSON Flat File (Project 1) | SQLite Database (Project 2) |
|---|---|---|
| Storage Format | Plain text JSON string | Binary file with 4KB pages & B-Tree indexes |
| Memory Usage | Loads whole file into RAM | Reads only requested pages from disk |
| Query Mechanism | Python loops (O(N) linear scan) | SQL queries (O(log N) indexed B-Tree) |
| Crash Safety | High risk of file corruption on crash | ACID compliant (Journal / WAL file) |
| Type Safety | Untyped JSON key-value pairs | Schema enforced (INTEGER, TEXT, REAL) |

## 2. The Pipeline: Connection, Cursor & Commit

**1. Connection (conn):** Represents the open file pipeline between Python in RAM and expenses.db on disk.
**2. Cursor (cursor):** The worker/pointer executing SQL strings, holding result buffers, and handling row iteration.
**3. Commit (conn.commit()):** Flushes changes from the temporary transaction staging buffer to permanent disk storage.

```
import sqlite3

# Context Manager automatically commits transactions and rolls back on exception
with sqlite3.connect("expenses.db") as conn:
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS expenses (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            item TEXT NOT NULL,
            amount REAL NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    ''')
```

## 3. Common Code Bugs & Fixes (Debugging Level 4)

• **Missing 'VALUES' keyword in INSERT:** INSERT INTO expenses (item, amount) VALUES (?, ?)

• **Interpolating raw strings (SQLi Risk):** Always use parameterized '?' placeholders: cursor.execute(query, (item, amount))

• **Unclosed parenthesis in cursor.execute:** Ensure tuples are properly enclosed: cursor.execute(sql, (param1, param2))

• **Forgetting conn.commit():** Without commit, changes sit in temporary staging and are discarded when connection closes.

## 4. Project 2 Production Reference Code

```
import sqlite3

def add_expense_db(item, amount):
    with sqlite3.connect("expenses.db") as conn:
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO expenses (item, amount) VALUES (?, ?)",
            (item, amount)
        )

def view_expenses_db():
    with sqlite3.connect("expenses.db") as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT id, item, amount, created_at FROM expenses")
        rows = cursor.fetchall()
        for row in rows:
            print(f"ID {row[0]}: {row[1]} - ${row[2]:.2f} (Added: {row[3]})")
```